CLA 4: ML Model Deployment Strategies and Model Serving using FastAPI

Dataset: Titanic – Machine Learning from Disaster (Kaggle train.csv)

Objective

To train and evaluate a machine-learning classification model using the Titanic dataset, save the complete pipeline, demonstrate online, batch and streaming inference, and serve predictions through a FastAPI REST API.

Task 1: Model Training

The Titanic dataset was loaded and preprocessed. A Random Forest classification pipeline was trained using a training split and evaluated on a test split. The complete preprocessing and model pipeline was saved as model.pkl.

Result: Test accuracy was 79.89%; model.pkl was saved successfully.

Task 2: Online Inference

The saved pipeline was loaded and used to predict the outcome for one passenger record.

Result: Prediction — Did not survive.

Task 3: Batch Inference

Twenty passenger records were passed to the saved pipeline together. The results were written to batch_predictions.csv.

Result: 20 predictions generated and saved.

Task 4: Streaming Inference

Ten records were processed one at a time with a one-second delay to simulate incoming streaming data.

Result: 10 records processed with a prediction displayed for each.

Task 5: Model Serving using FastAPI

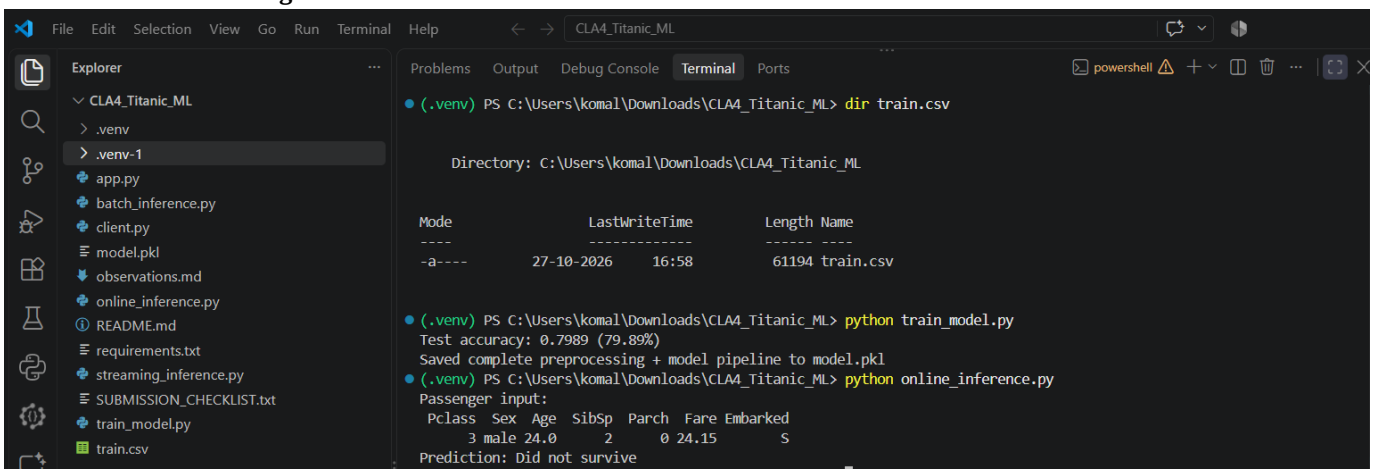
The FastAPI application loaded model.pkl and provided GET /health and POST /predict endpoints. Swagger UI (/docs) was used to test the health endpoint, a valid prediction, and an invalid request. client.py was also run against the API.

Result: /health returned HTTP 200 with model_loaded=true; valid /predict returned HTTP 200; invalid Pclass=8 returned HTTP 422; client.py returned Status: 200.

Execution Screenshots

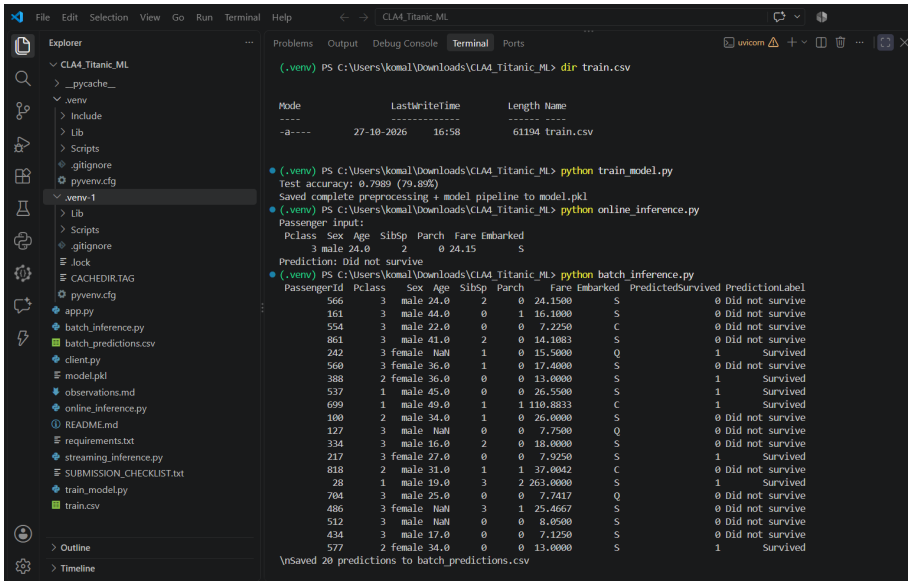
Screenshots are placed directly under their relevant task headings with minimal spacing.

Task 1: Model Training / Task 2: Online Inference



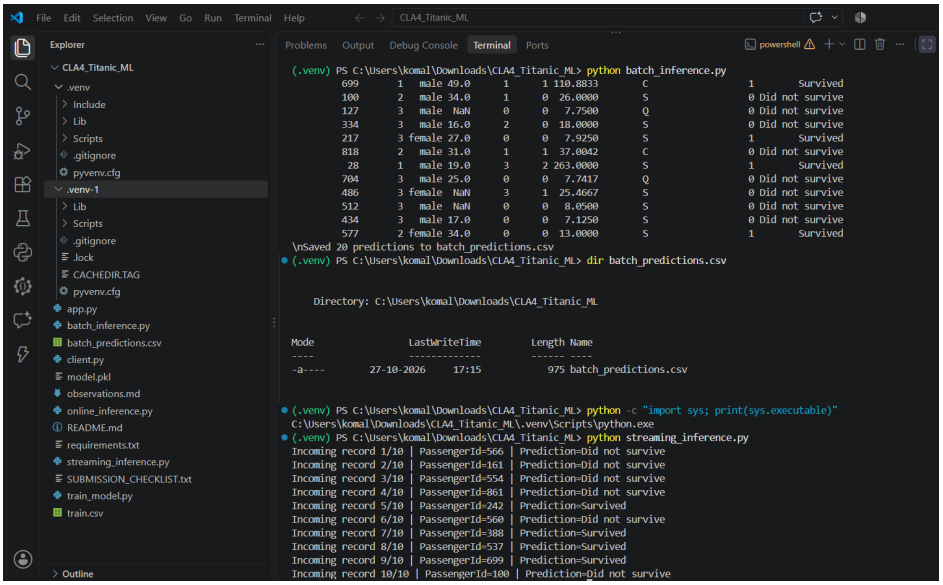
Model training achieved 79.89% test accuracy and saved model.pkl. The online inference script predicted "Did not survive."

Task 3: Batch Inference



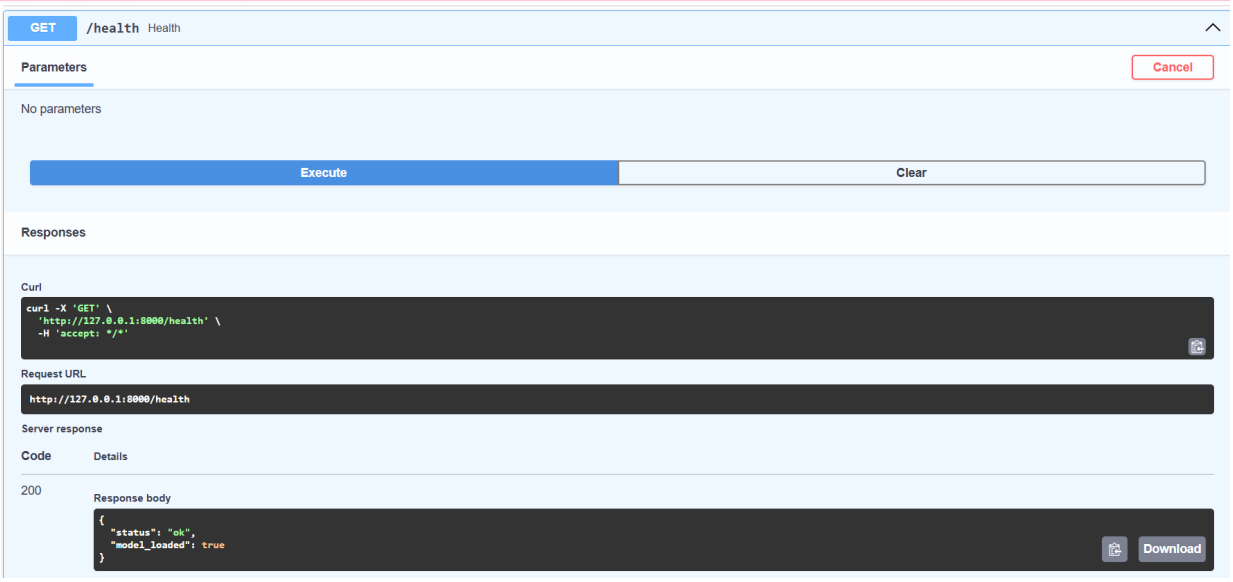
Twenty passenger records were predicted together and saved to batch_predictions.csv.

Task 4: Streaming Inference



Ten records were processed individually with a one-second delay.

Task 5: FastAPI Health Endpoint



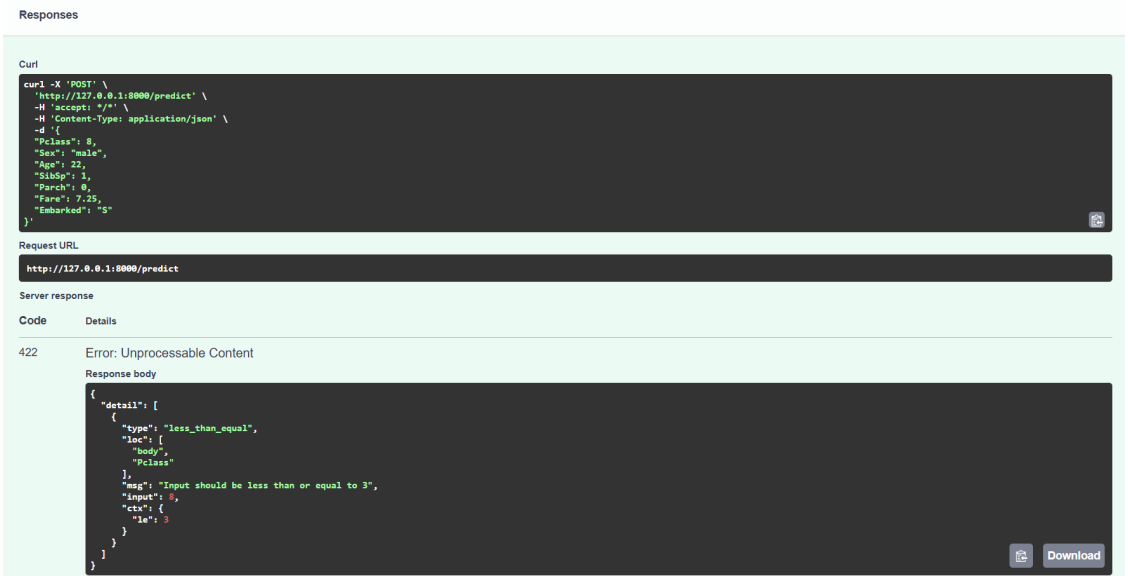
GET /health returned HTTP 200, status=ok and model_loaded=true.

Task 5: Valid Prediction Request



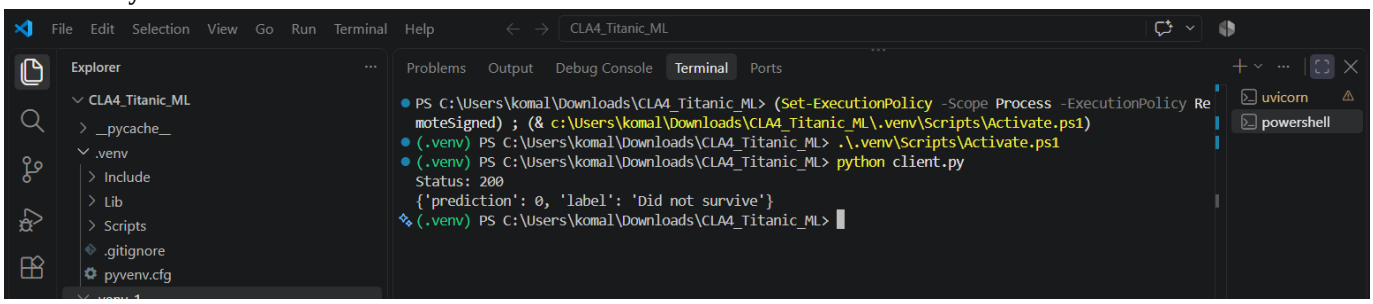
A valid POST /predict request returned HTTP 200 with prediction=0 and label=Did not survive.

Task 5: Invalid Request Validation



Pclass=8 returned HTTP 422 because Pclass must be between 1 and 3.

Task 5: Python Client



client.py returned Status: 200 with a prediction response.

Implementation and Results

Task	Implementation	Result
Model training	Trained and saved the Random Forest pipeline.	79.89% test accuracy; model.pkl saved.
Online inference	Predicted one passenger.	Did not survive.
Batch inference	Predicted 20 records together.	batch_predictions.csv generated.
Streaming inference	Processed 10 records with 1-second delay.	10 predictions displayed.
FastAPI health	Tested GET /health.	HTTP 200; model_loaded=true.
API validation	Tested valid and invalid POST /predict.	Valid: 200; invalid: 422.

Task	Implementation	Result
Python client	Executed client.py.	Status: 200.

Observations

1. Online Inference vs Batch Inference

Online inference predicts one record at a time; batch inference predicts a group of records together.

2. Streaming Inference vs Batch Inference

Streaming inference processes records individually as they arrive. Batch inference processes a predefined group together.

3. Why FastAPI?

FastAPI exposes the model through REST endpoints, Pydantic validates input data, and Swagger UI supports interactive testing.

4. Why reuse the same trained model?

Reusing one saved pipeline keeps preprocessing and predictions consistent and avoids retraining for each inference strategy.

Conclusion

The assignment demonstrated a saved ML pipeline, online, batch and streaming inference, and FastAPI model serving with request validation. The model achieved 79.89% test accuracy.